1) Variables

- Create a variable A that is equal to 7 and B that is equal to 5. Then write a script to swap the content of the two variables so that A is equal to 5 and B to 7 (so A takes the value of B and B of A)

In [1]:
```
A = 7
B = 5

C = A
A = B
B = C
print("A =", A)
print("B =", B)
print("C =", C)
```

```
A = 5
B = 7
C = 7
```

In [2]:
```
A = 7
B = 5

A, B = B, A
print("A =", A)
print("B =", B)
```

```
A = 5
B = 7
```

2) Operators

- Do an algorithm that convert a number of seconds entered by the user into "XX DAYS XX HOURS XX MIN XX SEC"

In [7]:
```
total = int(input("Enter a number of seconds: "))

days = total // (60 * 60 * 24)
remaining = total % (60 * 60 * 24)
hours = remaining // (60 * 60)
remaining = remaining % (60 * 60)
minutes = remaining // (60)
seconds = remaining % (60)
print(total, "s is : ", days, " Days, ", hours, " Hours, ", minutes, " Min, ", seconds, " Seconds.")
```

```
88458 s is :  1  Days,  0  Hours,  34  Min,  18  Seconds.
```

3) Logical operators

Considering the following values : A = 3, B = 9, C = False, D = NOT(C).

What are the results of the following operations :

1. (A > 8)
2. (B == 9)
3. NOT(A != 3)
4. NOT(C)
5. (A < B) OR C
6. NOT((A + B) != 12)
7. ((B == 5) OR ((c > 10) AND (a < 8)))
8. (((b == 5) OR ((c > 10) AND (a < 8))) OR (A < B) OR C) AND C

4) If/Elif/Else

- Create a small algorithm that takes a year as an input and tells if the year is a leap year or not. To recon if it is a leap year or not leap years are every 4 years, but not every 100 years except every 400 years.

In [13]:
```
year = int(input("Enter a year: "))

if year % 4 == 0 and (year % 100 != 0 or year % 400 == 0):
    print(year , " is a leap year.")
else:
    print(year , " is not a leap year.")
```

```
2000  is a leap year.
```

5) Loops

- Using loops, show the exponential of 2 from 2 to 1024. (there is an exponent operator **)
- Generate a number (you can generate random numbers with random.randint), and then ask the user to try to guess the number, while the user is incorrect, keep asking the number telling him if it is above or below.
- Create a calculator, that ask for two numbers and an operation and execute the right operation and show the result, after it ask the user if he wants to continue, if so ask for the numbers and operator else exit the loop.

In [15]:
```
for i in range(1, 11):
    print(2 ** i)

val = 2
for i in range(1, 11):
    print(2 * val)
```

```
2
4
8
16
32
64
128
256
512
1024
```

In [17]:
```
import random

rnd = random.randint(1, 100)
print(rnd)

user_in = -1
while user_in != rnd:
    user_in = int(input("Enter a number: "))
    if user_in < rnd:
        print("Too low")
    elif user_in > rnd:
        print("Too high")

print("You won")
```

```
44
Too high
Too low
You won
```

In [18]:
```
stop = False

while not stop:
    a = float(input("Enter a number: "))
    op = input("Enter an operator: ")
    b = float(input("Enter a number: "))

    if op == "+":
        print(a + b)
    elif op == "-":
        print(a - b)
    elif op == "*":
        print(a * b)
    elif op == "/":
        if a != 0:
            print(a / b)
        else:
            print("Cannot divide by zero")
    else:
        print("Invalid operator")

    stop_val = input("Do you want to stop? (y/n): ")
    if stop_val in "yY":
        stop = True

    # stop = True if stop_val in "yY" else False
```

```
97.0
```

Collection Basic

1. Create a list

Create a list called `fruits` with 3 fruits and print it.

In [20]:
```
fruits = ["apple", "banana", "cherry"]
```

2. Access an element

Given:

`numbers = [10, 20, 30]`
Print the first element.

In [19]:
```
numbers = [10, 20, 30]

numbers[0]
```

Out[19]:
```
10
```

3. Add an element to a list

Add `"orange"` to this list:

`fruits = ["apple", "banana"]`

In [22]:
```
fruits = ["apple", "banana"]

fruits.append("orange")

fruits
```

Out[22]:
```
['apple', 'banana', 'orange']
```

4. Loop through a list

Print each item in:

`colors = ["red", "green", "blue"]`

In [23]:
```
colors = ["red", "green", "blue"]

for item in colors:
    print(item)
```

```
red
green
blue
```

5. Length of a list

Print how many items are in:

`animals = ["cat", "dog", "rabbit"]`

In [24]:
```
animals = ["cat", "dog", "rabbit"]

print(len(animals))
```

```
3
```

6. Create a tuple

Create a tuple with 3 numbers and print it.

In [25]:
```
tpl = (1, 5, 8)
```

In [ ]:
```

```

7. Access tuple element

Given:

`t = (5, 10, 15)`
Print the second element.

In [27]:
```
t = (5, 10, 15)

tpl[1]
```

Out[27]:
```
5
```

8. Create a set

Create a set with numbers: 1, 2, 3 and print it.

In [28]:
```
st = { 1, 2, 3 }
```

9. Add to a set

Add the number `4` to this set:

`s = {1, 2, 3}`

In [29]:
```
st.add(4)

st
```

Out[29]:
```
{1, 2, 3, 4}
```

10. Simple dictionary

Create a dictionary with:

- key: `"name"` → value: your name
- key: `"age"` → value: your age

Then print the dictionary.

In [30]:
```
person = {
    "name" : "John",
    "age" : 25,
}

print(person)
```

```
{'name': 'John', 'age': 25}
```

Collection Medium

1. Count even numbers

Given:

`numbers = [1, 2, 3, 4, 5, 6]`
Count how many numbers are even and print the result.

In [31]:
```
numbers = [1, 2, 3, 4, 5, 6]

even_counter = 0
for elem in numbers:
    if elem % 2 == 0:
        even_counter += 1

print(even_counter)
```

```
3
```

2. Sum of positive numbers

`numbers = [-2, 5, -1, 7, -3]`
Calculate the sum of only the **positive numbers**.

In [32]:
```
numbers = [-2, 5, -1, 7, -3]
```

```
total = 0
for elem in numbers:
    if elem > 0:
        total += elem

print(total)
```

```
12
```

3. Find the largest number

`numbers = [3, 8, 2, 10, 5]`
Find and print the **largest number** without using `max()`.

In [37]:
```
numbers = [3, 8, 2, 10, 5]

# max(numbers)

max_nbr = numbers[0]

for elem in numbers:
    if elem > max_nbr:
        max_nbr = elem

for elem in numbers[1:]:
    if elem > max_nbr:
        max_nbr = elem

print(max_nbr)
```

```
10
```

4. Filter long words

`words = ["cat", "elephant", "dog", "giraffe"]`
Print only the words with **more than 3 letters**.

In [38]:
```
words = ["cat", "elephant", "dog", "giraffe"]

for elem in words:
    if len(elem) > 3:
        print(elem)
```

```
elephant
giraffe
```

5. Count occurrences

`letters = ["a", "b", "a", "c", "a"]`
Count how many times `"a"` appears.

In [39]:
```
letters = ["a", "b", "a", "c", "a"]

count_occur = {}

for elem in letters:
    if elem in count_occur.keys():
        count_occur[elem] += 1
    else:
        count_occur[elem] = 1

print(count_occur)
```

```
{'a': 3, 'b': 1, 'c': 1}
```

6. Remove duplicates (using a set)

`numbers = [1, 2, 2, 3, 4, 4]`
Create a new collection with **no duplicates** and print it.

In [41]:
```
numbers = [1, 2, 2, 3, 4, 4]

print(set(numbers))

arr = []
for elem in numbers:
    if not elem in arr:
```

```
        arr.append(elem)

print(arr)
```

```
{1, 2, 3, 4}
[1, 2, 3, 4]
```

7. Dictionary loop

`student = {"name": "Alice", "age": 20, "grade": 15}`
Loop through the dictionary and print:

```
key : value
```

In [42]:
```
student = {"name": "Alice", "age": 20, "grade": 15}

for key, val in student.items():
    print(key, ' -> ', val)
```

```
name  ->  Alice
age  ->  20
grade  ->  15
```

8. Count vowels in a string

`text = "hello world"`
Count how many vowels (`a, e, i, o, u`) are in the string.

In [44]:
```
text = "hello world"
vowels = ["a", "e", "i", "o", "u"]

vowels_count = 0
for c in text:
    if c in vowels:
        vowels_count += 1

print(vowels_count)
```

```
3
```

9. Find common elements

`list1 = [1, 2, 3, 4]`
`list2 = [3, 4, 5, 6]`
Print the numbers that appear in **both lists**.

In [46]:
```
list1 = [1, 2, 3, 4]
list2 = [3, 4, 5, 6]

common_elem = []
for elem in list1:
    for elem2 in list2:
        if elem == elem2:
            print(elem)
            common_elem.append(elem)

print(common_elem)

common_elem = []
for elem in list1:
    if elem in list2:
        common_elem.append(elem)

print(common_elem)
```

```
3
4
[3, 4]
[3, 4]
```

10. Simple grade checker

`grades = [12, 8, 15, 9, 18]`
For each grade:

- Print `"Pass"` if ≥ 10
- Print `"Fail"` if < 10

In [47]:
```
grades = [12, 8, 15, 9, 18]

for val in grades:
    if val >= 10:
        print("Pass")
    else:
        print("Fail")
```

```
Pass
Fail
Pass
Fail
Pass
```

Functions/Procedures

1. Divisibility check

Write a function:

`def is_multiple(a, b):`
Return `True` if `a` is divisible by `b`, otherwise `False`.

In [3]:
```
def is_multiple(a, b):
    if b == 0:
        return False

    return a % b == 0

is_multiple(5, 2)
```

Out[3]:
```
False
```

2. Sum of digits

`def digit_sum(n):`
Return the sum of all digits in the number.

e.g. 123 => 7

In [5]:
```
def digit_sum(n):
    total = 0

    while n > 0:
        total += n % 10
        n //= 10

    return total

def digit_sum2(n):
    total = 0
    as_str = str(n)
    for nbr in as_str:
        total += int(nbr)

    return total

digit_sum(123456789)
```

Out[5]:
```
45
```

3. Count vowels

`def count_vowels(text):`
Return the number of vowels in a string (`a, e, i, o, u`).

In [6]:
```
def count_vowels(text):
    vowels = "aeiouAEIOU"
    count = 0

    for c in text:
        if c in vowels:
            count += 1
    return count

count_vowels("Hello world!")
```

Out[6]: 3

4. Prime number check

`def is_prime(n):`
Return `True` if `n` is a prime number, otherwise `False`.

In [9]:
```
def is_prime(n):
    if n < 2:
        return False
    if n == 2:
        return True
    for i in range(3, n):
        if n % i == 0:
            return False
        return True

is_prime(15)
```

Out[9]: False

5. Multiplication table printer

`def print_mul_table(n):`
Print the multiplication table of `n` from 1 to 10.

In [10]:
```
def print_mul_table(n):
    for i in range (0, 10):
        print(n * (i + 1))

print_mul_table(5)
```

```
5
10
15
20
25
30
35
40
45
50
```

6. Reverse a number

`def reverse_number(n):`
Return the number reversed.

123 => 321

In [12]:
```
def reverse_number(n):
    reverse_nbr = ""

    str_val = str(n)
    for c in range(len(str_val) - 1, -1, -1):
        reverse_nbr += str_val[c]

    return int(reverse_nbr)

reverse_number(123)
```

Out[12]: 321

7. Password strength checker

`def check_password(password):`
Return:

- `"weak"` if length < 6
- `"medium"` if length is between 6 and 10
- `"strong"` if length > 10

In [13]:
```
def check_password(password):
    length = len(password)

    if length < 6:
        return "Weak"
```

```
    elif length <= 10:
        return "Medium"

    return "Strong"

print(check_password("asdf1234"))
```

```
Medium
```

8. Sum of squares

`def sum_squares(numbers):`
Return the sum of the squares of all numbers in the list.

In [14]:
```
def sum_squares(numbers):
    total = 0
    for num in numbers:
        total += num ** 2

    return total

sum_squares([12, 2, 3, 4, 5, 6, 7])
```

Out[14]:
```
283
```

9. Clean invalid scores

`def clean_scores(scores):`
Return a new list containing only values between **0 and 20 inclusive**.

In [15]:
```
def clean_scores(scores):
    new_scores = []
    for score in scores:
        if score >= 0 and score <= 20:
            new_scores.append(score)

    return new_scores

clean_scores([21, 15, -1, 5, 12, 16])
```

Out[15]:
```
[15, 5, 12, 16]
```

10. Find second largest

`def second_largest(numbers):`
Return the second largest number in the list.

In [16]:
```
def second_largest(numbers):
    unique = list(set(numbers))
    unique.sort()
    return unique[-2]

second_largest([1, 2, 3, 4, 5, 6, 7])
```

Out[16]:
```
6
```

11. Student result converter

`def student_report(students):`
Convert a dictionary of student grades into:

- `"Pass"` if grade ≥ 10
- `"Fail"` otherwise

```
{
    "alice": 12,
    "bob": 10,
    "charlie": 9
}
{
    "alice": "Pass",
    "bob": "Pass",
    "charlie": "Fail"
}
```

In [17]:
```
def student_report(students):
    result_dict = {}
```

```
    for key, val in students.items():
        result_dict[key] = "Pass" if val >= 10 else "Fail"

    return result_dict


students = {
    "alice": 12,
    "bob": 10,
    "charlie": 9
}
student_report(students)
```

Out[17]: {'alice': 'Pass', 'bob': 'Pass', 'charlie': 'Fail'}

12. Merge with sum

`def merge_and_sum(d1, d2):`
Merge two dictionaries:

- If a key exists in both, **sum the values**

e.g. { 'a': 2, 'b': 5, 'c': 2 } and { 'a': 3, 'd': 4 } => { 'a': 5, 'b': 5, 'c': 2, 'd': 4 }

In [18]:
```
def merge_and_sum(d1, d2):
    for key, val in d2.items():
        if key in d1:
            d1[key] += val
        else:
            d1[key] = val

    return d1

merge_and_sum({ 'a': 2, 'b': 5, 'c': 2 }, { 'a': 3, 'd': 4 })
```

Out[18]: {'a': 5, 'b': 5, 'c': 2, 'd': 4}

13. Email domain counter

`def count_domains(emails):`
Given a list of emails, return a dictionary counting how many emails belong to each domain.

```
email = [
    'test@gmail.com',
    'asdf@gmail.com',
    'zxcv@test.be',
    'qwer@asdf.nl'
]
```

will return

```
{
    "gmail.com": 2,
    "test.be": 1,
    "asdf.nl": 1,
}
```

In [19]:
```
def count_domains(emails):
    results = {}

    for email in emails:
        domain = email.split("@")[1]
        if domain not in results:
            results[domain] = 1
        else:
            results[domain] += 1

    return results

email = [
    'test@gmail.com',
    'asdf@gmail.com',
    'zxcv@test.be',
    'qwer@asdf.nl'
]

count_domains(email)
```

Out[19]: {'gmail.com': 2, 'test.be': 1, 'asdf.nl': 1}